II.1102 JAVA Programming and Algorithms
- Assisted project: Session 1 -

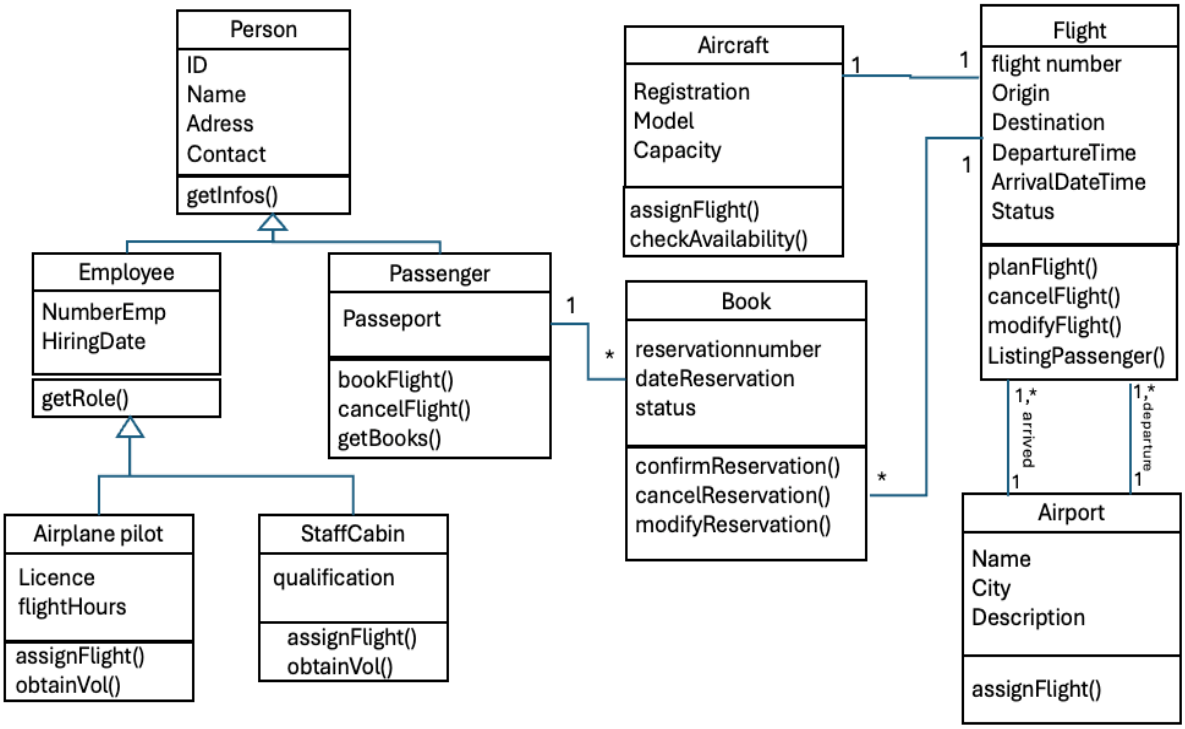
1. General information

The project will focus on the development of an application that manages an airline reservation system with all the required functionalities. More information on this subject in the following sections. Before describing the requirements for this project, here are a few general guidelines:

1. The project is carried out in pairs (possibly individually).
2. It is absolutely forbidden to use code generation tools. This will be considered as flagrant plagiarism.
3. Bonuses may be awarded for original features of your choice.

2. Project overview: Airline management application

A Java-based airline reservation system that uses object-oriented programming. The system can manage customers, crew, reservations and cancellations. You will implement the various classes from the diagram shown in the figure below, using OOP concepts such as inheritance, encapsulation, association and composition.



3. Project Functionalities

Based on the diagram above, you will create all the classes, considering the schematic relational constraints (inheritance, associations, etc.).

In each class, you should add CRUD methods for adding, modifying, retrieving and deleting object instances.

For each class you create, you need to implement the various methods shown in the diagram. The main methods to be implemented are as follow :

- **Personal management:**
  - **GetInfos()** : It's a method for displaying various information about a person.
  - **GetRole()** : this method returns an employee's role by identifier (pilot, cabin crew, etc.).
- **Reservations management:**
  - **bookFlight() :** a method triggered by the 'Passenger' class. A passenger can book several flights. A list of reserved flights must be added to the reservation class.
  - **cancelBook ():** This method allows you to cancel a reservation given by its ID (reservation number).
  - **getReservations() :** This method is used to obtain a list of information about a reservation given by its ID (reservation number).
- **Flight management :**
  - **assignFlight()** : A crew must be assigned to each flight. A pilot as well as a cabin crew must ensure the routing of the flight.
  - **getFlight()** : This method allows you to obtain a list of information on a flight given by its ID (flightNumber).
  - **planFlight()** : The company's activity must be planned. A set of flights will be added over a given period (a one-day schedule, for example)
  - **cancelFlight() :** This method is used to cancel a flight given by its ID (flight number).

- **Aircraft management :**
  - **affectFlight()** : For each flight, we assign an aircraft from the list of available aircraft.
  - **checkAvailability()** : Before assigning an aircraft to a given flight, you need to check its availability in the scheduled timetable.

4. Advanced features (Bonus)

- **Statistics and reports:**

  - Generate reports on number of flights, passengers carried, revnue generated.
  - Display of most popular destinations.

5. Technical constraints

- The application must be developed in Java using object-oriented programming concepts,
- Use of appropriate data structures for information storage (arrays, lists, etc.),
- Work with a hosting service for projects under development: **GitHub**
- Use a tool for managing and automating the production of Java software projects: **Maven**

6. Useful links:

- Github tutorial with Intellij Idea:
  https://www.jetbrains.com/help/idea/github.html

- How to configure a project with GitHub:

https://www.jetbrains.com/help/idea/manage-projects-hosted-on-github.html

- Maven tutorial on Intellij Idea:

  https://www.jetbrains.com/help/idea/maven-support.html

- maven installation video on Intellij Idea :

https://www.jetbrains.com/guide/java/tutorials/marco-codes-maven/installing-maven/

7. Deliverables

Deliverables to be submitted on Moodle before the deadline:

- **A compressed file** containing all your written code **(Deadline: December 5).**